

INSTITUTO FEDERAL DE ALAGOAS
CAMPUS PALMEIRA DOS ÍNDIOS
CURSO TÉCNICO INTEGRADO AO ENSINO MÉDIO DE INFORMÁTICA

AMANDA BISPO BARROS DE ARAÚJO
MAISA BEZERRA AZARIAS

Pesquisa Técnica sobre Validação de Dados em Sistemas Web

PALMEIRA DOS ÍNDIOS, AL
2026

AMANDA BISPO BARROS DE ARAÚJO
MAISA BEZERRA AZARIAS

PESQUISA TÉCNICA SOBRE VALIDAÇÃO DE DADOS EM SISTEMA WEB

Trabalho elaborado na disciplina de programação web para obtenção de nota bimestral, solicitado pelo professor Carlos Jean Costa Cabral.

PALMEIRA DOS ÍNDIOS, AL
2026

Sumário

Sumário.....**2**
Introdução.....**3**
1. Validação Client-Side vs. Server-Side.....3
1.1. Validação no Lado do Cliente (Client-Side).....3
1.2. Validação no Lado do Servidor (Server-Side).....3
2. Atributos de Validação Nativa (HTML5).....4
2.1. Atributo required.....4
2.2 Atributo pattern.....4
2.3. Seletores de tamanho minlenght e maxlenght.....4
3. Estados Visuais e Estilização Avançada com CSS.....**5**
3.1. A Pseudo-classe :invalid.....5
3.2 A Pseudo-classe :valid.....5
3.3 Aplicação Técnica e Boas Práticas de Design.....5
Conclusão.....**6**
Referências.....**7**

Introdução

Este documento apresenta a fundamentação teórica que ampara a transição do portal para um ecossistema ativo. Nele, detalham-se os conceitos fundamentais de validação de dados nas vertentes Client-Side e Server-Side, o uso semântico e declarativo de atributos nativos da especificação HTML5, e os mecanismos de feedback estético fornecidos pelas pseudo-classes estruturais do CSS.

1. Validação Client-Side vs. Server-Side

A validação de dados é o processo arquitetural que garante que a entrada de dados fornecida pelo usuário atenda estritamente às regras de negócio e restrições técnicas preestabelecidas antes de ser processada ou armazenada. Esse processo ocorre fundamentalmente em duas frentes independentes, porém complementares:

1.1. Validação no Lado do Cliente (Client-Side)

Ocorre diretamente no ambiente de execução do navegador do usuário, antes mesmo que os dados sejam despachados pela rede através de uma requisição HTTP. Ela é implementada através dos recursos declarativos do HTML5 e de regras imperativas programadas em JavaScript.

- **Vantagens e Foco:** Seu principal objetivo é fornecer uma rica Experiência do Usuário (User Experience). O feedback instantâneo — impedindo o envio do formulário no momento do clique se houver falhas — previne a frustração de recarregamentos desnecessários da página e economiza largura de banda e processamento no servidor.

- **Limitações Cruciais:** Não oferece segurança à aplicação. Como o código Client-Side roda sob controle total do usuário, qualquer indivíduo com conhecimento básico pode desativar o JavaScript nas configurações do navegador ou burlar as travas do HTML5 manipulando o DOM através do painel de ferramentas do desenvolvedor (F12).

1.2. Validação no Lado do Servidor (Server-Side)

Ocorre de forma centralizada nos servidores da aplicação (Back-end) assim que os parâmetros da requisição HTTP são recebidos. É codificada em linguagens de programação de servidor como PHP, Node.js, Python, Java ou C#

- **Vantagens e Foco (Segurança):** É a **única linha de defesa verdadeiramente segura** de uma aplicação web. Ela protege o banco de dados contra a inserção de lixo, inconsistências operacionais e tentativas de invasão ou exploração de

vulnerabilidades perigosas, tais como *SQL Injection* (Injeção de comandos SQL estruturados) e *Cross-Site Scripting (XSS)*.

- **Limitações Operacionais:** Como exige que os dados viajem através da rede até o servidor e aguardem o processamento e o retorno da resposta HTTP, ela apresenta maior latência (tempo de espera), o que pode prejudicar a fluidez caso seja usada como única interface de aviso ao usuário.

2. Atributos de Validação Nativa (HTML5)

Atributos de validação nativa são comandos ou regras de segurança que você escreve direto dentro das tags HTML.

Eles são chamados de **"nativos"** porque já vêm embutidos no próprio motor dos navegadores (Chrome, Firefox, Safari, Edge). Isso significa que o próprio navegador assume a responsabilidade de fiscalizar o que o usuário está digitando e bloqueia o envio do formulário automaticamente se as regras forem descumpridas, sem que você precise escrever nenhuma linha de código JavaScript para fazer essa checagem básica.

A especificação moderna do HTML5 introduziu mecanismos declarativos robustos que permitem ao navegador realizar validações estruturais pesadas sem que o desenvolvedor precise redigir blocos complexos de código imperativo em JavaScript. Os três principais atributos obrigatórios para a atividade são:

2.1. Atributo required

Ele define que o campo é obrigatório para o envio do formulário. Caso o usuário tente enviar a página com esse campo em branco, o navegador bloqueia o envio imediatamente e exibe um balão de alerta nativo instruindo-o a preencher o que falta.

2.2 Atributo pattern

Esse atributo serve para **impor um formato obrigatório** para o preenchimento do campo através de Expressões Regulares (Regex). Ele define exatamente quais caracteres e em qual sequência eles devem ser digitados. No caso do telefone brasileiro, por exemplo, o padrão pattern="\([0-9]{2}\)\s[0-9]{4,5}-[0-9]{4}" exige que o texto tenha o DDD entre parênteses, um espaço em branco, de 4 a 5 dígitos, um hífen e mais 4 dígitos no final. Qualquer digitação fora desse padrão deixa o campo inválido automaticamente.

2.3. Seletores de tamanho minlenght e maxlenght

Controlam a contagem quantitativa de caracteres inseridos dentro de inputs de texto ou áreas de texto expandidas (<textarea>). O minlength impede o envio se o comprimento do texto for inferior ao limite definido (essencial para evitar respostas vagas ou nulas em campos de ouvidoria, exigindo, por exemplo, o mínimo de 15 caracteres estipulado na atividade), enquanto o maxlength atua de forma preventiva no teclado do usuário, travando fisicamente digitações que extrapolem o tamanho limite permitido no banco de dados.

3. Estados Visuais e Estilização Avançada com CSS

A validação declarativa do HTML5 ganha contornos estéticos e profissionais por meio da integração nativa com o motor de renderização do CSS através de pseudo-classes de validação estrutural. As principais ferramentas de estilização reativa são:

3.1. A Pseudo-classe :invalid

É um estado que o navegador ativa automaticamente no campo se qualquer uma das regras de validação for descumprida. Isso permite que o desenvolvedor crie estilos visuais em tempo real, como mudar a cor da borda, deixando bem claro para o usuário que aquele campo está com erro.

3.2 A Pseudo-classe :valid

Funciona exatamente ao contrário da anterior. Sempre que o campo estiver preenchido corretamente e cumprir todas as exigências (como um e-mail válido com arroba e domínio), o navegador assume esse estado. Isso serve para dar um retorno visual positivo ao usuário, mudando a cor do campo para verde, por exemplo.

3.3 Aplicação Técnica e Boas Práticas de Design

O recomendado é usar essas pseudo-classes junto com o seletor :focus para garantir uma boa experiência de design. Exibir alertas visuais de erro assim que a página carrega assusta o visitante e prejudica a usabilidade. Por isso, o ideal é que os estilos de erro só apareçam na tela quando o usuário estiver de fato interagindo e editando o campo.

```css
/* Aplica borda vermelha e fundo suave enquanto o usuário edita e o dado estiver incorreto */
.campo-grupo input:focus:invalid,
.campo-grupo textarea:focus:invalid {
    border: 2px solid #e74c3c;
    background-color: #fdf2f2;
}

/* Aplica borda verde suave e confirmação visual assim que o campo torna-se válido */
.campo-grupo input:valid,
.campo-grupo textarea:valid {
    border: 2px solid #2ecc71;
    background-color: #f4fbf7;
}
```

Essa automação garante que a interface responda em tempo real conforme o usuário digita, ajudando-o a preencher os dados corretamente. Além disso, simplifica o código da aplicação, pois elimina completamente a necessidade de criar scripts em JavaScript para adicionar ou remover classes de estilo manualmente no HTML (DOM).

Conclusão

A transição de um portal passivo para um ecossistema interativo exige uma compreensão nítida das engrenagens de validação. A validação no lado do cliente (*Client-Side*), amparada pelos atributos nativos do HTML5 e pelo JavaScript, cumpre com excelência o papel de guiar o visitante em tempo real, otimizando a usabilidade (UX) e poupando recursos de rede.

Contudo, conforme analisado, ela jamais deve ser a única barreira de proteção de um sistema, papel este que pertence exclusivamente à validação no lado do servidor (*Server-Side*) para garantir a segurança contra vulnerabilidades. Conclui-se que o uso inteligente de pseudo-classes CSS (`:valid` e `:invalid`) integradas ao comportamento do formulário enriquece a interface sem sobrecarregar o código com scripts desnecessários, resultando em uma aplicação web que equilibra perfeitamente uma ótima experiência de uso com integridade técnica.

Referências

MDN WEB DOCS. *Validação de formulário de dados*. Mozilla Developer Network. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn/Forms/Form_validation. Acesso em: 17 de julho de 2026.
WHATWG. *HTML Living Standard: Shadows of the UI - Form Validation*. Web Hypertext Application Technology Working Group. Disponível em: https://html.spec.whatwg.org/. Acesso em: 17 de julho de 2026.
W3C. *CSS Selector Level 4: UI Pseudo-classes (:valid and :invalid)*. World Wide Web Consortium. Disponível em: https://www.w3.org/TR/selectors-4/. Acesso em: 17 de julho de 2026.
FLANAGAN, David. **JavaScript: O Guia Definitivo**. Porto Alegre: Bookman, 2021.